VMES Grinds

1544. Make The String Great

1544. Make The String Great

Solved

Easy | Topics | Companies | Hint

Given a string `s` of lower and upper case English letters.

A good string is a string which doesn't have **two adjacent characters** `s[i]` and `s[i + 1]` where:

- `0 <= i <= s.length - 2`
- `s[i]` is a lower-case letter and `s[i + 1]` is the same letter but in upper-case or **vice-versa**.

To make the string good, you can choose **two adjacent** characters that make the string bad and remove them. You can keep doing this until the string becomes good.

Return *the string* after making it good. The answer is guaranteed to be unique under the given constraints.

Notice that an empty string is also good.

Example 1:

```
Input: s = "leEeetcode"
Output: "leetcode"
Explanation: In the first step, either you choose i = 1 or i = 2, both
will result "leEeetcode" to be reduced to "leetcode".
```

Example 2:

```
Input: s = "abBAcC"
Output: ""
Explanation: We have many possible scenarios, and all lead to the same
answer. For example:
"abBAcC" --> "aAcC" --> "cC" --> ""
"abBAcC" --> "abBA" --> "aA" --> ""
```

Example 3:

```
Input: s = "s"
Output: "s"
```

Constraints:

- `1 <= s.length <= 100`
- `s` contains only lower and upper case English letters.

Explanation

Goal

What makes a string "Bad"?

- A string is bad if it contains two adjacent characters that are:
- The same letter (e.g., 'e' and 'E').
- But different cases (lower vs. upper).
- Goal: Remove these pairs recursively until the string is "Good".

Why a Stack?

The problem is recursive. Removing a pair might bring two new characters together that previously weren't adjacent.

A Stack is the perfect tool for this:

- We iterate through the string.
- We always compare the current character against the top of the stack (the last surviving character).
- If they conflict, we Pop (destroy both).
- If they don't, we Push

The Solution

```python
class Solution:
    def makeGood(self, s: str) -> str:
        stack = []
        for char in s:
            if stack and stack[-1] != char and stack[-1].lower() == char.lower():
                stack.pop()
            else:
                stack.append(char)

        return ''.join(stack)
```

Time and Space Complexity Analysis

```python
class Solution:
    def makeGood(self, s: str) -> str:
        stack = []
        for char in s:
            if stack and stack[-1] != char and stack[-1].lower() == char.lower():
                stack.pop()
            else:
                stack.append(char)

        return ''.join(stack)
```

Space - O(n)

Time - O(n)

Key Takeaway

VMES Grinds MMM
Chee sar

Whenever a problem involves "pairwise destruction" or processing elements based on immediate proximity, a Stack is usually the optimal choice.